

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÀI TẬP LỚN
MÔN HỆ NHÚNG

Đề tài: XÂY DỰNG TRÒ CHƠI GALAGA
TRÊN KIT PHÁT TRIỂN STM32F429I-DISC1

Giảng viên hướng dẫn: TS. Ngô Lam Trung

Lớp: 157539

Nhóm sinh viên thực hiện

STT	Họ và tên	MSSV
1	Đặng Tuấn Anh	20225247
2	Đỗ Xuân Huy	20225331
3	Nguyễn Quang Huy	20210430

Hà Nội, tháng 6 năm 2025

MỤC LỤC

I. Giới thiệu đề tài.....	2
1. Mô tả về sản phẩm.....	2
2. Yêu cầu hệ thống.....	2
II. Thiết kế.....	3
1. Thiết kế phần cứng.....	3
2. Thiết kế phần mềm.....	7
3. Sơ đồ khối hệ thống.....	14
III. Cài đặt hệ thống.....	15
1. Phân chia công việc.....	15
2. Kết quả đạt được.....	16
3. Đánh giá.....	17

I. Giới thiệu đề tài

1. Mô tả về sản phẩm

Đề tài tập trung xây dựng một trò chơi **Galaga** - một game bắn súng dạng **shoot-em-up**, trong đó người chơi điều khiển một phi thuyền không gian di chuyển sang trái hoặc phải để né tránh hỏa lực của kẻ thù và tiêu diệt các đội hình quái vật ngoài hành tinh đang tấn công từ phía trên màn hình để sống sót lâu nhất có thể vào danh điểm cao.

2. Yêu cầu hệ thống

2.1. Yêu cầu chức năng

Chức năng hiển thị đồ họa:

- Giao diện trò chơi được hiển thị trên LCD với các thành phần chính: tàu người chơi, kẻ địch, đạn, hiệu ứng nổ, điểm số.
- Hiển thị điểm số và màn hình Game Over khi trò chơi kết thúc.

Chức năng điều khiển:

- Joystick có thể điều khiển tàu di chuyển sang trái hoặc phải theo ý người chơi.
- Nút nhấn có thể được sử dụng để bắn đạn lên phía trên.

Chức năng âm thanh:

- Tích hợp phát âm thanh khi bắn, khi tiêu diệt kẻ địch hoặc khi người chơi bị tấn công..

2.1. Yêu cầu phi chức năng

Tính ổn định:

- Game phải hoạt động ổn định trong thời gian dài, không bị treo hoặc lỗi logic.

Tính mượt mà:

- Chuyển động của các đối tượng (tàu, đạn, địch) phải mượt mà, không bị giật.

Tính phản hồi nhanh:

- Phản hồi từ joystick và nút bấm phải gần như tức thời.
- Phát âm thanh đồng bộ với hành động.

Tính hoàn thiện phần cứng:

- Phần cứng được bố trí trực quan và gọn gàng.
- Các kết nối phần cứng phải chắc chắn, hạn chế nhiễu tín hiệu.

II. Thiết kế

1. Thiết kế phần cứng

1.1. Hệ thống gồm các thành phần chính:

- Kit STM32F429I-DISC1
- Mini USB-cable
- Nút bấm
- Joystick
- Loa

1.2. Kit STM32F429I-DISC1

Dòng sản phẩm MCU hiệu suất cao STM32 F4 dựa trên ARM Cortex-M4 với các lệnh DSP và FPU là phần mở rộng của danh mục STM32 hàng đầu trong ngành hướng tới hiệu suất cao hơn nữa. Tần số hoạt động lên đến 180 MHz. ST Microelectronics STM32F429I-DISC1 là Discovery board dành cho các bộ vi điều khiển (MCU) STM32F429. Nó bao gồm mọi thứ cần thiết cho người mới bắt đầu và người dùng có kinh nghiệm để bắt đầu phát triển ứng dụng với giao diện người dùng đồ họa.



Hình 1 Kit phát triển STM32F429I-DISCO

1.3. Sơ đồ ghép nối

1.3.1. Joystick

Joystick	Kit
GND	GND
+5V	3V
VRx	PC3(ADC1_IN13)
VRy	-
SW	-

Sử dụng ngắt để xử lý tín hiệu giữa joystick và vi xử lý, khi người dùng di chuyển joystick, ngắt được kích hoạt thì sẽ gửi dữ liệu qua hàng đợi ≈ 60 lần 1 giây.

1.3.2. Nút bấm

Nút bấm	Kit
PIN 1	GND
PIN 2	PG2

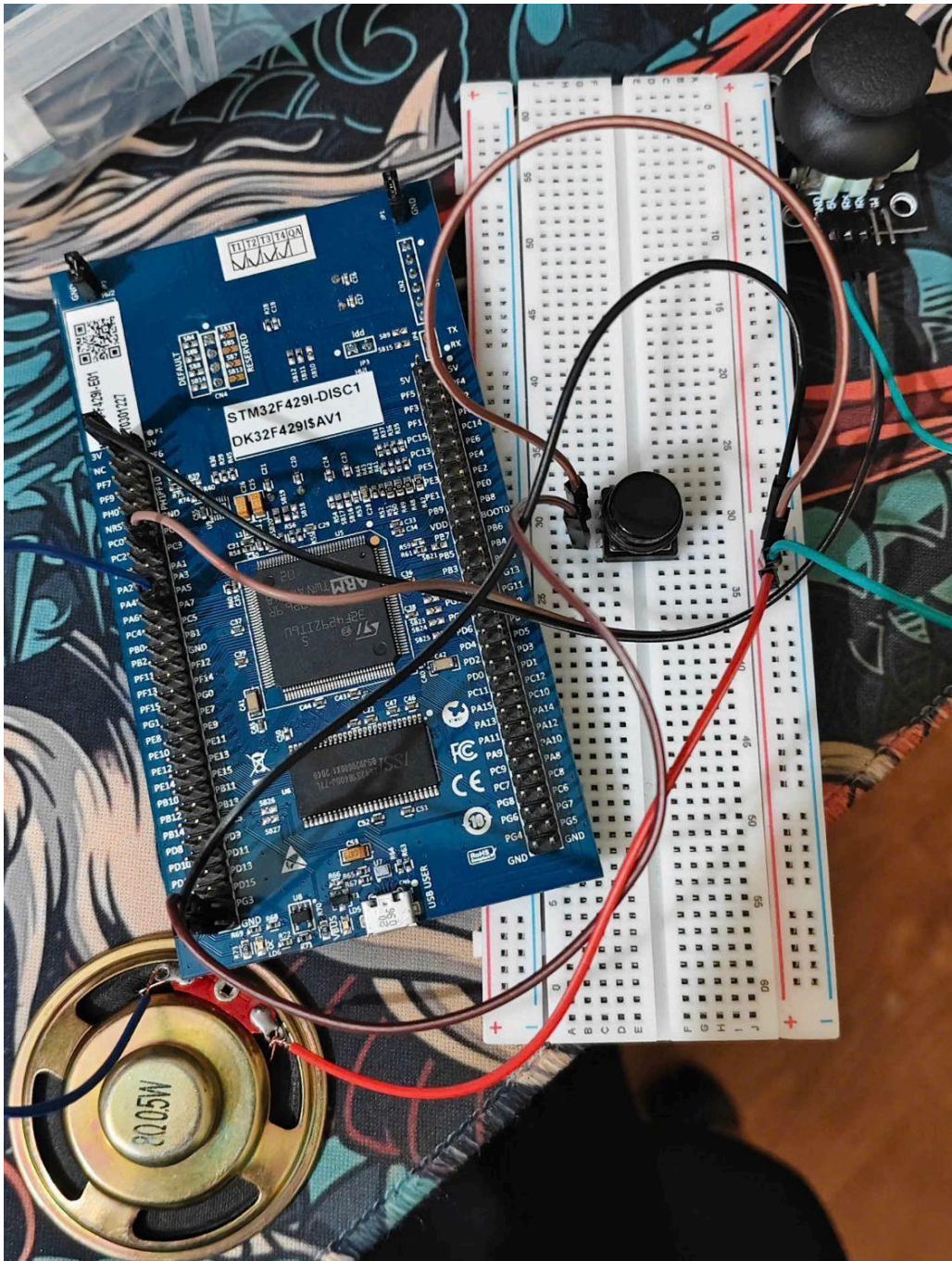
Polling trạng thái nút bấm ≈ 60 lần 1 giây.

1.3.3. Loa

Loa	Kit
+	PA7
-	GND

Truyền bằng PWM với tần số 8kHz

1.4. Sơ đồ mạch



Hình 2 Sơ đồ mạch

2. Thiết kế phần mềm

2.1. Các module

2.1.1. Module xử lý input từ joystick và nút bấm

```
osMessageQueueId_t Queue1Handle;  
osMessageQueueId_t Queue2Handle;  
osMessageQueueId_t Queue3Handle;  
  
const osMessageQueueAttr_t Queue1_Attributes = { .name = "Queue1" };  
const osMessageQueueAttr_t Queue2_Attributes = { .name = "Queue2" };  
const osMessageQueueAttr_t Queue3_Attributes = { .name = "Queue3" };  
uint8_t joyX;  
uint8_t checkADC1;  
/* USER CODE FOR GUI */
```

Khai báo hàng đợi dữ liệu cho input, và biến *joyX* để lưu giá trị x của joystick, biến *checkADC1* để xử lý ngắt.

```
checkADC1 = 0;  
HAL_ADC_Start_IT(&hadc1);
```

Khởi tạo biến *checkADC1 = 0* và khởi tạo ngắt cho ADC1

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {  
    if (hadc == &hadc1) {  
        joyX = HAL_ADC_GetValue(&hadc1);  
        checkADC1=1;  
    }  
}
```

Hàm callback để gọi khi có ngắt xảy ra, lưu giá trị đọc được của chân ADC1 vào *joyX* đồng thời đặt giá trị biến *checkADC1 = 1* để xử lý ở hàm *StartDefaultTask()*.

```

void StartDefaultTask(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for (;;) {
        if (checkADC1 == 1) {
            if (joyX > 220) {
                uint8_t data = 'R';
                osMessageQueuePut(Queue1Handle, &data, 0, 10);
            } else if (joyX < 30) {
                uint8_t data = 'L';
                osMessageQueuePut(Queue2Handle, &data, 0, 10);
            }
            checkADC1 = 0;
            HAL_ADC_Start_IT(&hadc1);
        }
        if (HAL_GPIO_ReadPin(GPIOG, GPIO_PIN_2) == GPIO_PIN_RESET) {
            uint8_t data = 'F';
            osMessageQueuePut(Queue3Handle, &data, 0, 10);
        }
        osDelay(16);
    }
    /* USER CODE END 5 */
}

```

Ở hàm *StartDefaultTask()* kiểm tra *checkADC* khi bằng 1 tức là có ngắt xảy ra, thực hiện kiểm tra giá trị x của joystick (giá trị x nằm trong khoảng từ 0 đến 255) và đẩy dữ liệu vào hàng đợi tương ứng với vị trí joystick. Sau đó, đặt lại giá trị *checkADC = 1* và bắt đầu lại ADC.

Hàm còn liên tục polling trạng thái chân PG2 được kết nối với nút bấm và gửi dữ liệu vào hàng đợi.

Sử dụng *osDelay(16)* để đồng bộ với *GameScreenView* vì chương TouchGFX hoạt động ở 60 FPS tương ứng với 16.67 ms một lần cập nhật.

2.1.2. Xử lý và hiển thị trò chơi

```
#include "Enemy.hpp"
#include "Bee.hpp"
#include "Butterfly.hpp"
#include "Boss.hpp"

#include "Bullet.hpp"
#include "Entity.hpp"
#include <cmsis_os.h>
#include <Ship.hpp>

struct SpawnRequest {
    enum Type { BOSS, BEE, BUTTERFLY } type;
    int x, y;
};
#define MAX_SPAWN_QUEUE 32
#define SCREEN_WIDTH 240
#define ENEMY_WIDTH 16
class Game {
public:
    Game();
    ~Game();
    void update();
    Ship ship;
    Boss bosses[MAX_BOSS];
    Bee bees[MAX_BEE];
    Butterfly butterflies[MAX_BUTTERFLY];
    Bullet eBullets[MAX_EBULLET];
    uint32_t score;
    uint32_t stage;
    void queueSpawn(SpawnRequest::Type type, int x, int y);
    void handleSpawnQueue();
private:
    int count;
    SpawnRequest spawnQueue[MAX_SPAWN_QUEUE];
    int spawnQueueSize = 0;
    int spawnDelayCounter = 0;
};
extern Game game;
extern void spawnStage(int stage);
extern void GameThread(void *argument);
#endif
```

Tạo lớp game để quản lý các đối tượng như tàu, kẻ thù, đạn, và các trạng thái của game. Hàm *update()* là được gọi định kỳ để cập nhật logic trò chơi: kiểm tra va chạm, cập nhật trạng thái các đối tượng.

Khai báo ngoại đối tượng game *extern Game game* để sử dụng trong GameScreenView

```
void GameScreenView::setupScreen() {
    GameScreenViewBase::setupScreen();

    const osThreadAttr_t update_attributes = { .name = "update", .stack_size =
        8192 * 2, .priority = (osPriority_t) osPriorityNormal, };

    gameTaskHandle = osThreadNew(GameThread, NULL, &update_attributes);
}
```

Hàm *GameThread()* là một hàm để chạy dưới dạng một luồng (thread) của FreeRTOS thông qua *cmsis_os*. Hàm này chạy song song với luồng xử lý giao diện GameScreenView với *osDelay(16)* đảm bảo đồng bộ giữa cập nhật logic trò chơi liên tục và độc lập với giao diện, giúp giao diện không bị đơ, lag khi xử lý logic phức tạp.

2.1.3. Lưu điểm cao nhất

```
//highest score
if (game.score > highestScore) {
    highestScore = game.score;
    uint32_t hScore = game.score;
    for (int i = MAX_SCORE_LEN - 1; i >= 0; i--) {
        uint8_t digit = hScore % 10;
        hScore /= 10;
        highScoreImages[i].setBitmap(touchgfx::Bitmap(DIGIT_BITMAPS[digit]));
        highScoreImages[i].setVisible(true);
        if(hScore==0) break;
    }
}
```

Khi điểm hiện tại *game.score* cao hơn điểm cao nhất *highestScore*, cập nhật *highestScore* = *game.score*

```

#include "flash_score.h"
#include "stm32f4xx_hal.h"

#define FLASH_SECTOR_SCORE FLASH_SECTOR_11
#define FLASH_SCORE_ADDRESS 0x080E0000 // Sector 11 start
#define SCORE_FLASH_MAGIC 0xA5A5A5A5 // Dùng để xác nhận dữ liệu hợp lệ

typedef struct {
    uint32_t magic;
    uint32_t score;
} FlashScore_t;

static FlashScore_t* flash_score_ptr = (FlashScore_t*)FLASH_SCORE_ADDRESS;

```

Lưu điểm cao nhất vào Flash sector 11

```

void Flash_SaveHighScore(uint32_t score) {
    HAL_FLASH_Unlock();

    // Xóa sector 11
    FLASH_EraseInitTypeDef eraseInit;
    uint32_t sectorError;

    eraseInit.TypeErase = FLASH_TYPEERASE_SECTORS;
    eraseInit.Sector = FLASH_SECTOR_SCORE;
    eraseInit.NbSectors = 1;
    eraseInit.VoltageRange = FLASH_VOLTAGE_RANGE_3; // 2.7V - 3.6V

    if (HAL_FLASHEx_Erase(&eraseInit, &sectorError) != HAL_OK) {
        HAL_FLASH_Lock();
        return; // Lỗi khi xóa
    }

    // Ghi magic + score
    HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, FLASH_SCORE_ADDRESS, SCORE_FLASH_MAGIC);
    HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, FLASH_SCORE_ADDRESS + 4, score);

    HAL_FLASH_Lock();
}

```

Hàm Flash_SaveHighScore xóa dữ liệu cũ, ghi dữ liệu mới. Hàm được gọi khi game over để tránh giật lag.

```
uint32_t Flash_LoadHighScore(void) {
    if (flash_score_ptr->magic == SCORE_FLASH_MAGIC) {
        return flash_score_ptr->score;
    } else {
        return 0; // Nếu chưa có gì thì coi như điểm là 0
    }
}
```

Hàm Flash_LoadHighScore dùng để lấy điểm cao nhất

2.1.4. Xử lý âm thanh

```
SoundPlayer soundPlayer = {nullptr, nullptr, 0, 0, 0, false};
```

Khởi tạo một biến soundPlayer với các giá trị mặc định ban đầu cho struct SoundPlayer.

```
struct SoundPlayer {
    const uint32_t* tones;
    const uint16_t* durations;
    uint8_t count;
    uint8_t index;
    uint16_t tickCounter;
    bool isPlaying;
};

void GameScreenView::startTone(uint32_t freq) {
    uint32_t period = 1000000 / freq;
    __HAL_TIM_SET_AUTORELOAD(&htim3, period - 1);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, period / 2);
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2);
}

void GameScreenView::stopTone() {
    HAL_TIM_PWM_Stop(&htim3, TIM_CHANNEL_2);
}

void GameScreenView::playToneSequence(const uint32_t* tones, const uint16_t* durations, uint8_t count) {
    soundPlayer.tones = tones;
    soundPlayer.durations = durations;
    soundPlayer.count = count;
    soundPlayer.index = 0;
    soundPlayer.tickCounter = 0;
    soundPlayer.isPlaying = true;
    startTone(tones[0]);
}
```

StartTone bắt đầu phát âm thanh bằng PWM với tần số được chỉ định trên Timer 3 kênh 2.

StopTone dừng phát âm thanh bằng cách tắt PWM trên Timer.

PlayToneSequence khởi tạo và bắt đầu phát một chuỗi âm thanh gồm nhiều tần số và thời lượng khác nhau.

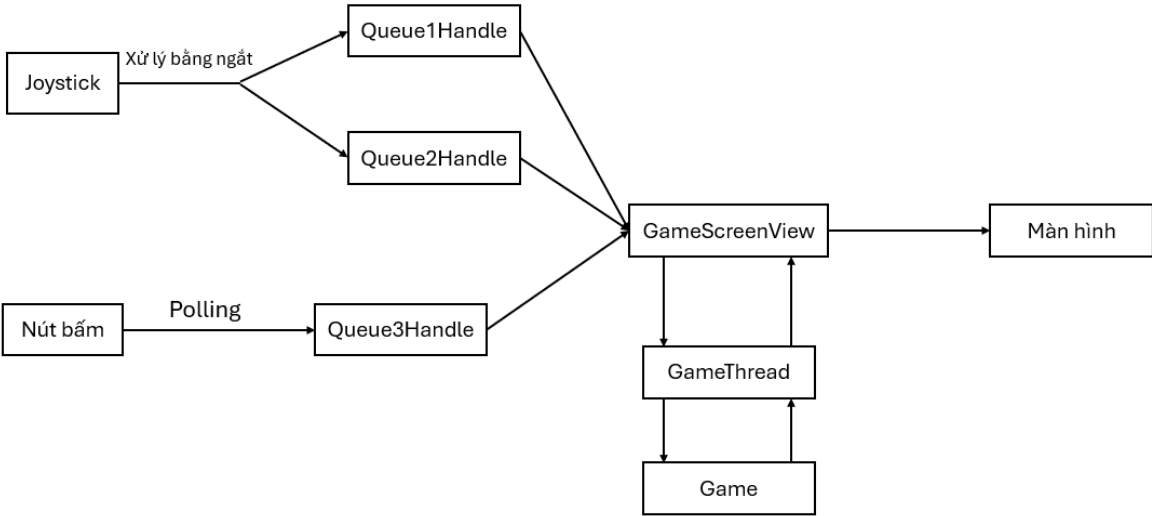
```
if (soundPlayer.isPlaying) {  
    soundPlayer.tickCounter++;  
    if (soundPlayer.tickCounter >= soundPlayer.durations[soundPlayer.index]) {  
        soundPlayer.index++;  
        soundPlayer.tickCounter = 0;  
        if (soundPlayer.index < soundPlayer.count) {  
            startTone(soundPlayer.tones[soundPlayer.index]);  
        } else {  
            stopTone();  
            soundPlayer.isPlaying = false;  
        }  
    }  
}
```

Kiểm tra nếu đang phát âm thì tăng bộ đếm thời gian cho tone hiện tại.

Nếu thời gian đủ, chuyển sang tone tiếp theo hoặc dừng phát nếu hết chuỗi.

Gọi startTone() cho âm mới hoặc stopTone() khi kết thúc toàn bộ.

3. Sơ đồ khối hệ thống



Hình 2 Sơ đồ khối hệ thống

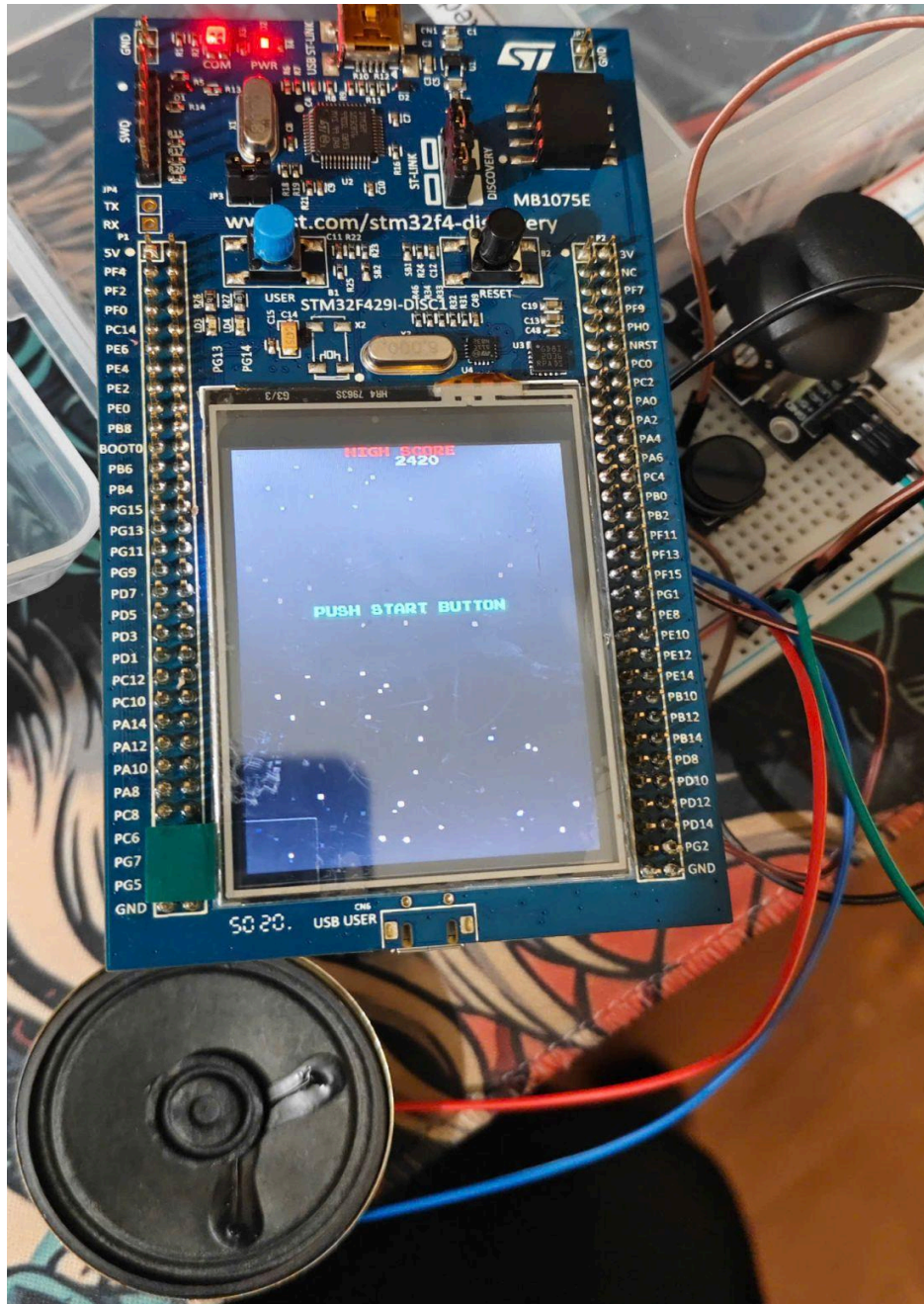
III. Cài đặt hệ thống

Mã nguồn project: <https://github.com/doxhuy2104/Galaga-STM32F429>

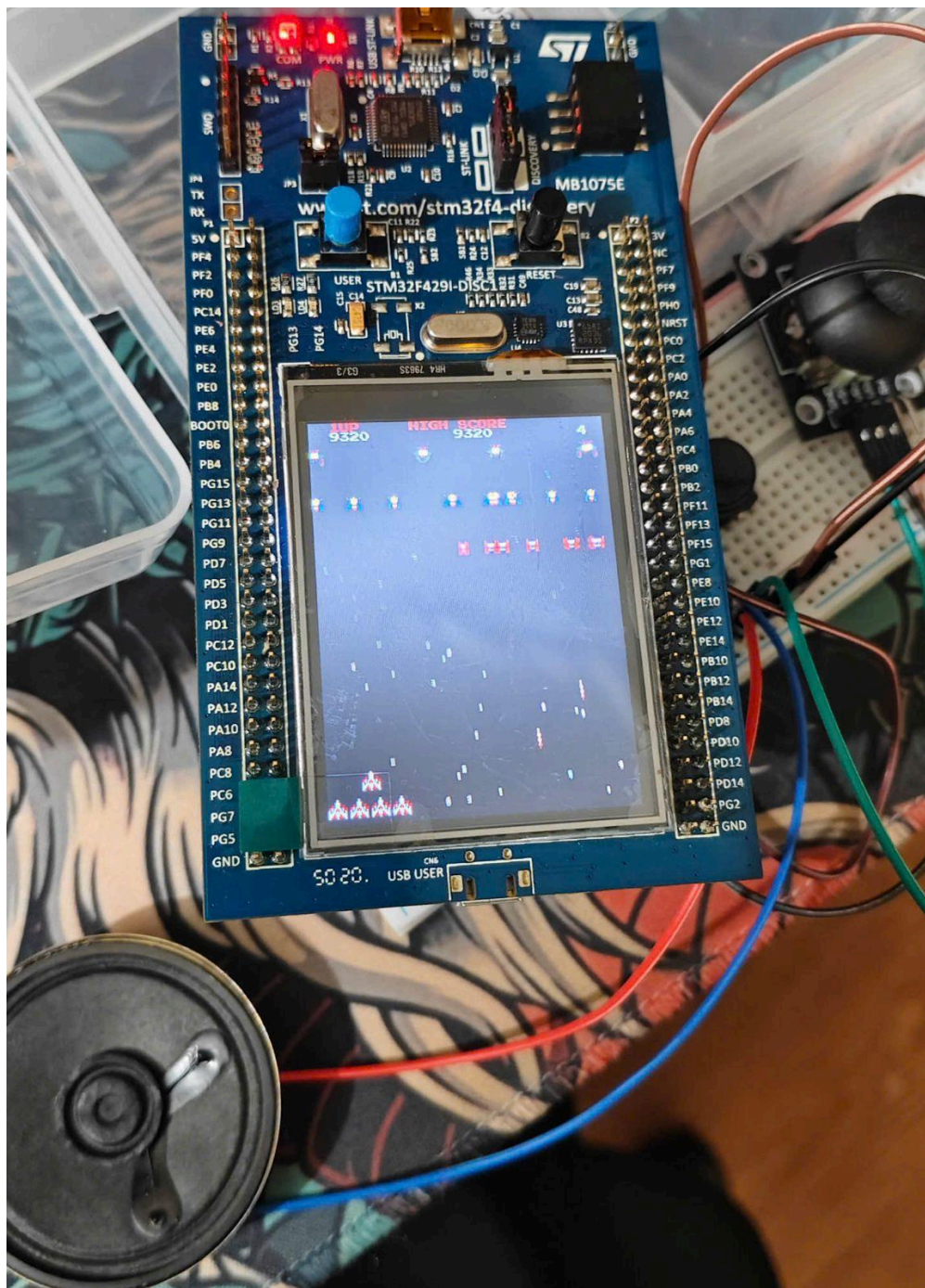
1. Phân chia công việc

Thành viên	Công việc
Đỗ Xuân Huy	Xử lý tín hiệu nút bấm và joystick, thiết kế giao diện, xây dựng và xử lý logic game, viết báo cáo.
Đặng Tuấn Anh	Thiết kế giao diện, xử lý logic game, viết báo cáo
Nguyễn Quang Huy	Thiết kế phần cứng âm thanh, xử lý logic âm thanh trong game, viết báo cáo.

2. Kết quả đạt được



Hình 4 Giao diện màn hình chính



Hình 6 Giao diện trò chơi

3. Đánh giá

3.1. Mức độ hoàn thành

Đáp ứng đầy đủ yêu cầu chức năng:

- Di chuyển, bắn, địch, va chạm, điểm số, giao diện, âm thanh.

Đạt được nhiều yêu cầu phi chức năng:

- Hệ thống hoạt động mượt mà, ổn định.
- Âm thanh tương tác đồng bộ với hành động.

3.2. Ưu điểm:

- Giao diện bắt mắt, dễ sử dụng.
- Phản hồi nhanh, không bị delay khi chơi.
- Âm thanh tăng tính hấp dẫn.

3.3. Nhược điểm:

- Lối chơi đơn giản, chưa hấp dẫn.
- Không có AI cho địch (di chuyển theo pattern cố định).
- Âm thanh đơn giản (1–2 hiệu ứng, chưa có nhạc nền).